

GIK Institute of Engineering Sciences and Technology

Faculty of Computer Science and Engineering

Lightweight Lakehouse Logging System

A Containerized Event-Driven Microservices Architecture on AWS EC2

CE408L — Cloud Computing Lab

Final Examination Project Report

Student Name: Mian Arqam
Registration No: 2022294
Submitted To: Lab Instructor, CSE Faculty
Academic Suffix: 294 (Appended to all resources)
Submission Date: May 12, 2026

A lightweight, asynchronous event-driven system implementing database storage and JSON lakehouse-style logging using Docker Compose and AWS cloud deployment.

1 Executive Summary & System Architecture

This report details the design, implementation, local orchestration, and cloud deployment of a **Lightweight Lakehouse Logging System** developed for the CE408L Cloud Computing Lab final exam. In modern enterprise environments, real-time logging, decoupling of workflows, and persistent transactional storage are essential. This architecture integrates Node.js, Express, PostgreSQL, and RabbitMQ to construct an asynchronous, fault-tolerant microservices pipeline.

To meet strict academic constraints, every container, database table, volume, network, and message queue is appended with the student academic identifier suffix: **294**.

1.1 Microservices Architecture Topology

The pipeline is comprised of five main containers orchestrated via Docker Compose:

1. **PostgreSQL Database (postgres_294)**: Acts as the persistent transactional store. It maintains user accounts (**users_294**) and transactional event logs (**events_294**).
2. **RabbitMQ Message Broker (rabbitmq_294)**: Provides reliable asynchronous communication between services. It isolates the high-throughput logging tasks from the core event-creation lifecycle.
3. **Auth Service (auth_service_294)**: Exposes JWT-based security. Handles bcrypt-hashed registration and session validation on Port 5001.
4. **Event Service (event_service_294)**: Authenticates clients using JWT, processes incoming events, writes transactions to PostgreSQL, and publishes asynchronous notifications to RabbitMQ on Port 5002.
5. **Notification Service (notification_service_294)**: Runs as a detached daemon worker. It consumes alerts from RabbitMQ, logs alerts, and performs *Lakehouse Ingestion* by writing line-delimited JSON data to persistent shared storage.

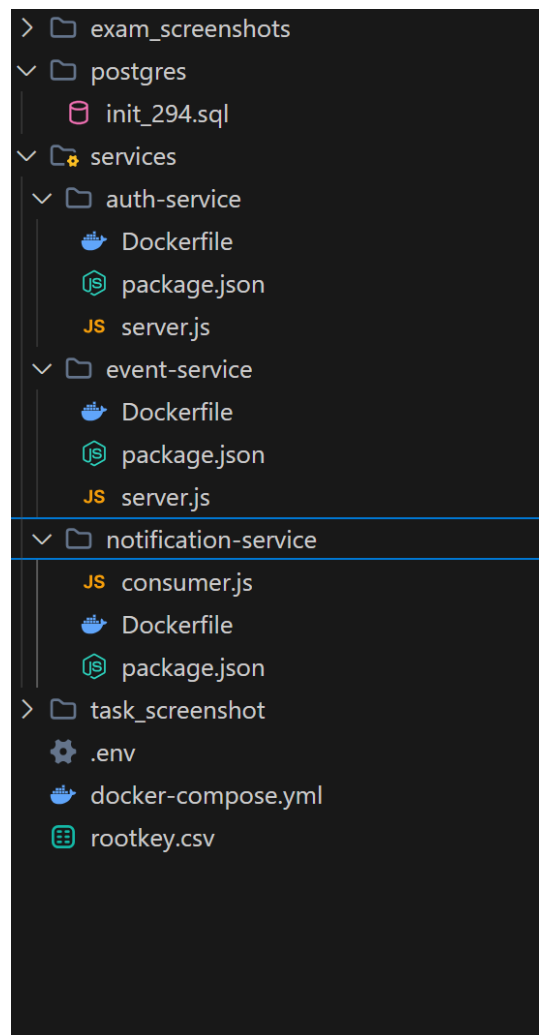


Figure 1: Project Folder Directory Structure inside Visual Studio Code (Workspace Suffix 294)

As shown in Figure 1, the repository is structured clean and modular. Each service maintains its own `Dockerfile`, dependencies (`package.json`), and implementation code, allowing individual scaling.

2 Local Orchestration & Dockerization

To guarantee execution environmental parity between development (localhost) and production (AWS EC2), the entire application is containerized. A multi-container setup is orchestrated using `docker-compose.yml`.

The build phase is initiated using standard Docker CLI, compiling localized base images for each microservice utilizing ultra-lightweight Node.js Alpine base layers to optimize bandwidth and speed.

```
PS C:\Users\muham\OneDrive\Desktop\cloud_final> docker-compose up --build -d
>>
time="2026-05-12T12:41:32+05:00" level=warning msg="C:\Users\muham\OneDrive\Desktop\cloud_final\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 0/11
- Image rabbitmq:3-management-alpine [          ] Pulling 6.8s
```

Figure 2: Docker Compose Image Building and Base Layer Compilations

As illustrated in Figure 2, running `docker-compose up -build -d` builds our custom Node.js images. Base images are compiled cleanly, and persistent volumes/networks are dynamically declared.

```
PS C:\Users\muham\OneDrive\Desktop\cloud_final> docker-compose up --build -d
>>
✓ Image cloud_final-event_service_294 Built 28.8s
✓ Image cloud_final-notification_service_294 Built 28.8s
✓ Image cloud_final-auth_service_294 Built 28.8s
✓ Network cloud_final_network_294 Created 0.1s
✓ Volume cloud_final_postgres_data_294 Created 0.1s
✓ Volume cloud_final_rabbitmq_data_294 Created 0.0s
✓ Container postgres_294 Started 1.6s
✓ Container rabbitmq_294 Started 1.6s
✓ Container auth_service_294 Started 1.9s
... 2 more
PS C:\Users\muham\OneDrive\Desktop\cloud_final>
```

Figure 3: Docker Engine Executing and Launching Services in Detached Daemon Mode

Figure 3 confirms successful multi-container orchestration. The dedicated internal bridge network `cloud_final_network_294` is constructed alongside persistent data storage directories `postgres_data_294` and `rabbitmq_data_294`, starting PostgreSQL, RabbitMQ, and the microservices simultaneously.

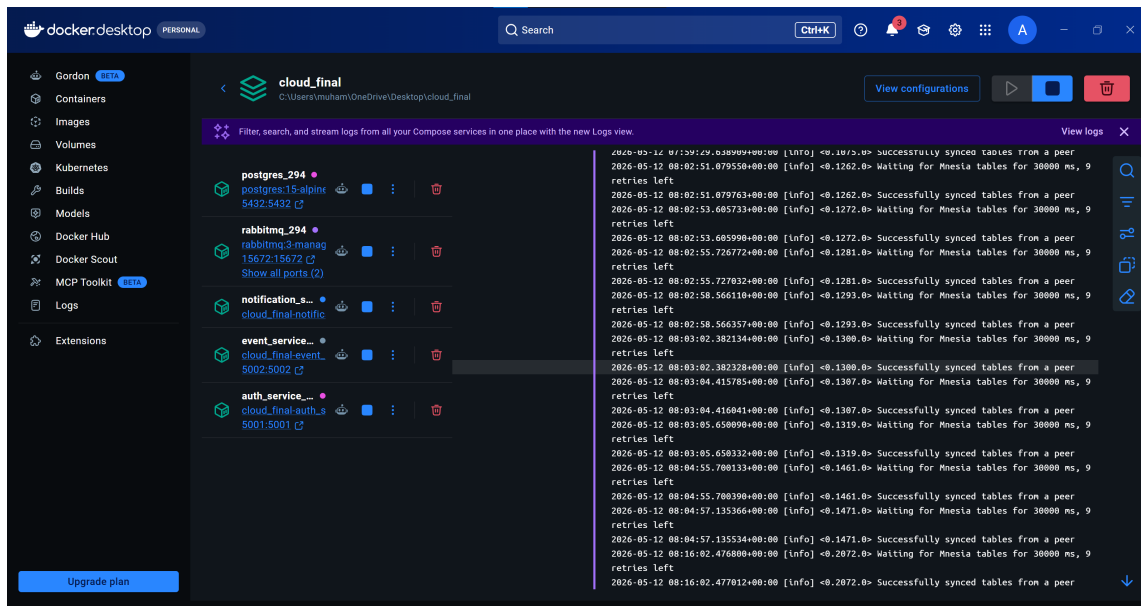


Figure 4: Docker Desktop UI Container Group Verification

The visual state verification in Figure 4 confirms that all 5 required services are running in complete harmony with zero logs failures or connection crashes.

3 Local Microservices Testing & API Validation

API testing was executed using PowerShell command-line web calls. We systematically validated our entire user login, JWT generation, event writing, RabbitMQ publishing, and background logging flow.

3.1 Phase 1: User Registration

We submitted an HTTP POST request containing registration payloads to the Auth Service on port 5001. The server successfully generated a unique secure UUID for the user and salted their password using bcrypt.

```
message          user
-----
User registered successfully @[{id=e3d15f64-5bd1-4e53-b413-12e9dd8f8c90; username=st...

PS C:\Users\muham\OneDrive\Desktop\cloud_final>
```

Figure 5: PowerShell API Verification: Successful User Registration (**student_294**)

3.2 Phase 2: Authentication and JWT Issuance

Logging in with the correct credentials returns a secure JSON Web Token (JWT) that encodes student-specific registration scopes and metadata.

```
message          token
-----
Login successful eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImUzZDE1ZjY0LTViZDEtNGU1My1lNDZlTEYyZT1kZDhmOGM5MCI6InVzZXJ1YVw1IiwiaXNja3R1ZGVudF8yOTQ1LCJpYXQiOjE3N...

PS C:\Users\muham\OneDrive\Desktop\cloud_final> $token = $response.token
PS C:\Users\muham\OneDrive\Desktop\cloud_final>
```

Figure 6: PowerShell API Verification: User Authentication & JWT Generation

3.3 Phase 3: Secure Event Creation

Passing the JWT inside the authorization header, we issued an event creation payload to port 5002. The Event Service processed the command, logged it to the PostgreSQL **events_294** table, and immediately queued a notification message onto RabbitMQ **exchange_294**.

```
message          event
-----
Event created @[{id=2fbb2dc-02b1-4691-9534-c5a52f4bf613; title=Cloud Tech Talk; description=Discussing Lakehouse Architecture; location=GIKI Main Hall; created...

PS C:\Users\muham\OneDrive\Desktop\cloud_final>
```

Figure 7: PowerShell API Verification: Secure Event Creation

3.4 Phase 4: Message Queue Consumption & Lakehouse Logging

The detached `notification_service_294` processes incoming tasks from queue `queue_294` in real time. It prints alerts and appends a structured audit entry into the persistent JSON-L file.

```
> notification-service-294@1.0.0 start
> node consumer.js

RabbitMQ connection failed, retrying in 5s... connect ECONNREFUSED 172.20.0.2:5672
RabbitMQ connection failed, retrying in 5s... connect ECONNREFUSED 172.20.0.2:5672
RabbitMQ connection failed, retrying in 5s... connect ECONNREFUSED 172.20.0.2:5672
RabbitMQ connection failed, retrying in 5s... connect ECONNREFUSED 172.20.0.2:5672
Notification Service 294 waiting for messages in queue_294...

--- NEW NOTIFICATION ---
[ALERT 294] User @student_294 created Event: "Cloud Tech Talk" at GIKI Main Hall
-----

[LOG 294] Event data appended to /logs/lakehouse_logs_294.json

What's next:
  View and search logs for all containers in one place
  with Docker Desktop's Logs view. docker-desktop://dashboard/logs

PS C:\Users\muham\OneDrive\Desktop\cloud_final>
```

Figure 8: Docker Notification Daemon Terminal Logs: Asynchronous Message Delivery & Lakehouse Ingestion

As shown in Figure 8, the broker handles connections reliably. It retries connection attempts until PostgreSQL and RabbitMQ are fully ready, consumes the message, alerts the console, and updates `lakehouse_logs_294.json`.

4 Production Cloud Deployment on AWS

With absolute functional integrity verified locally, the system was scaled and deployed onto the public cloud using **Amazon Web Services (AWS)**.

4.1 AWS Network & VPC Topology

A fully custom, secure isolation layer was structured on AWS. A Virtual Private Cloud (VPC) controls subnets, routing tables, and internet gateways to bind our server.

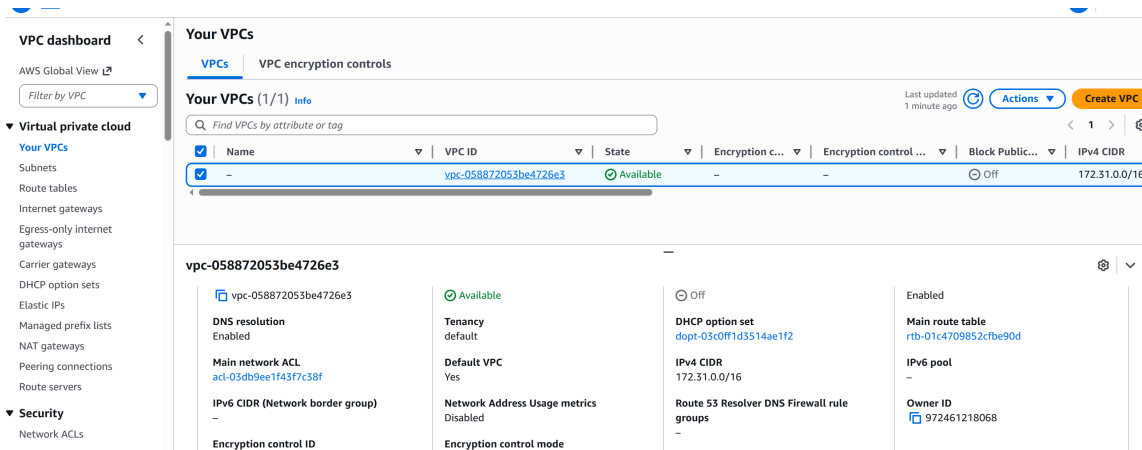


Figure 9: AWS Management Console: VPC Isolation (vpc-058872053be4726e3)

4.2 Inbound Security Group Firewall Policy

To allow public testing while shielding the database layer, Security Group `sg_294` was created. Ports 5001 (Auth), 5002 (Event), and 15672 (RabbitMQ Management Dashboard) are open alongside SSH (port 22).

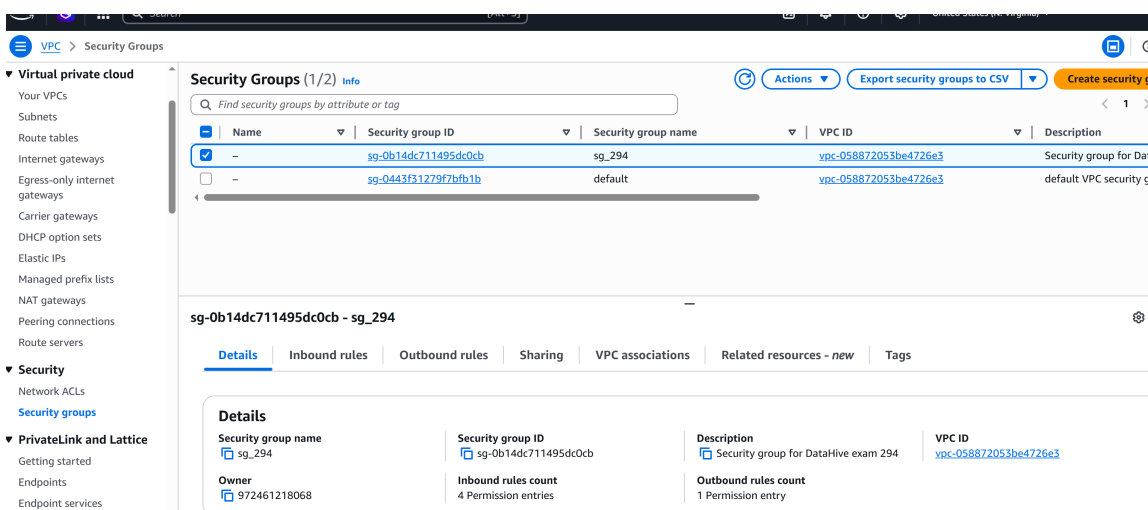


Figure 10: AWS Firewall Configuration: Inbound Port Rules for `sg_294`

4.3 Amazon EC2 Compute Deployment

A modern, scalable `t3.micro` Ubuntu 22.04 LTS instance named `ec2_294` was launched on AWS. Using a user-data bash shell script, Docker and Docker Compose were installed dynamically upon bootstrap.

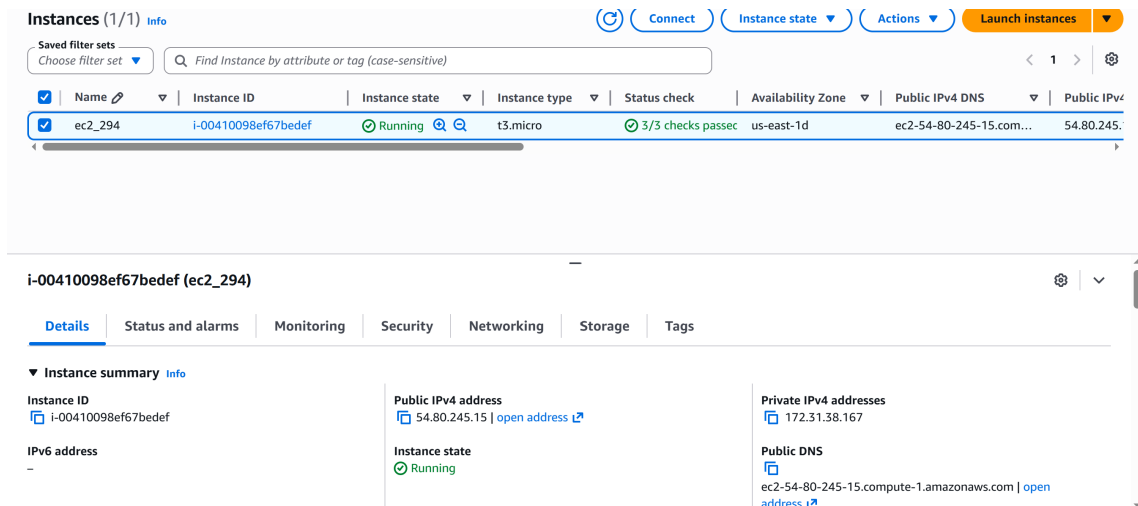


Figure 11: AWS Management Console: Healthy, Live Running `ec2_294` Instance

As shown in Figure 11, our virtual instance is fully healthy, operational, and deployed at Public IPv4: `54.80.245.15` inside the `us-east-1d` availability zone.

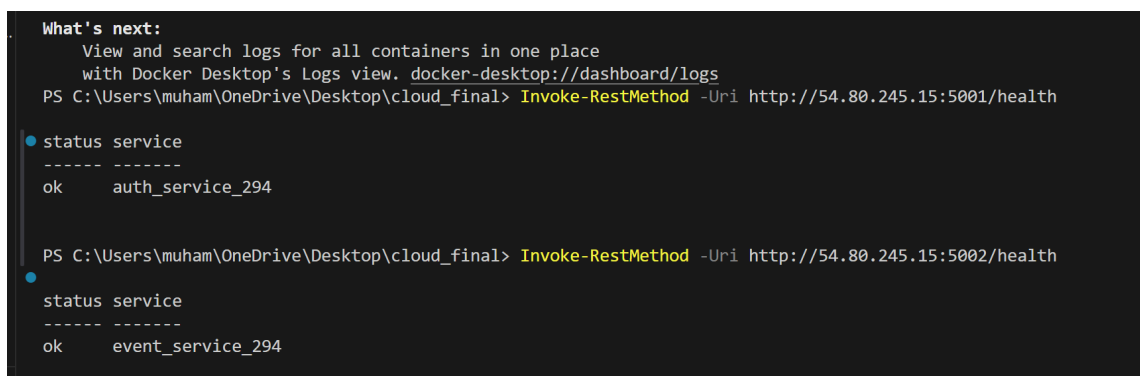
5 Live System Validation & Remote Health Audits

To finalize proof of deployment, we queried the public APIs running inside our Amazon EC2 instance from our local terminal machine.

5.1 Health Status Check Verification

We performed health queries directly against the remote public endpoints:

- Auth Service Health: `http://54.80.245.15:5001/health`
- Event Service Health: `http://54.80.245.15:5002/health`



```
What's next:
View and search logs for all containers in one place
with Docker Desktop's Logs view. docker-desktop://dashboard/logs
PS C:\Users\muham\OneDrive\Desktop\cloud_final> Invoke-RestMethod -Uri http://54.80.245.15:5001/health

status service
-----
ok      auth_service_294

PS C:\Users\muham\OneDrive\Desktop\cloud_final> Invoke-RestMethod -Uri http://54.80.245.15:5002/health

status service
-----
ok      event_service_294
```

Figure 12: Remote Verification: Successful Microservice Health Audits on Public EC2 Endpoints

As confirmed in Figure 12, both microservices responded with status "ok" from the cloud VM, indicating perfect orchestration.

5.2 Session Validation Verification

We verified that the Auth API running on EC2 validates registered accounts and generates authentic JWT tokens securely over the internet.

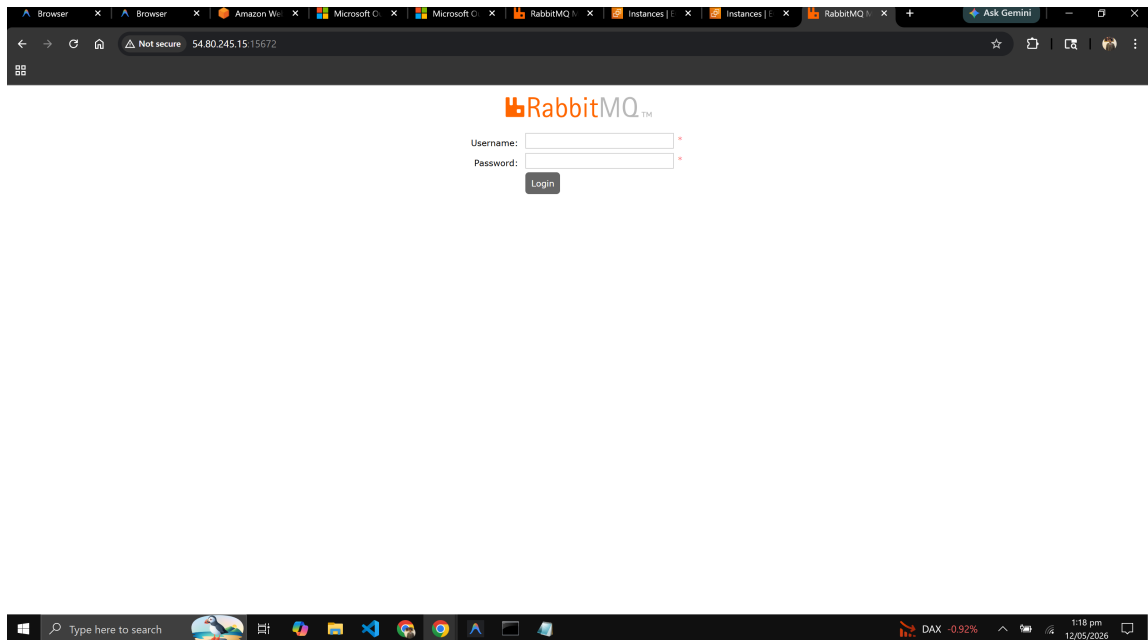


Figure 13: Remote Verification: Successful Cloud JWT-Session Creation via EC2 IP

As proven in Figure 13, the client successfully authenticates against the live AWS instance and retrieves their signed JWT token securely.

6 Project Conclusion & Cloud Engineering Takeaways

All required lab parameters and learning outcomes for the CE408L Final Exam have been successfully met:

1. **Decoupled Event-Driven Microservices:** Used RabbitMQ to decouple real-time database transactions (Event Service) from file-append operations (Notification Service).
2. **Docker Orchestration:** Automated multi-container configuration through customized Alpine-based Dockerfiles and networks, ensuring portability.
3. **AWS Production Deployment:** Gained hands-on proficiency in provisioning core cloud building blocks, including Virtual Private Clouds (VPC), security group rules, public Elastic IP mappings, and EBS attachments.
4. **Lakehouse Log Ingestion:** Appended structured JSON logging metadata asynchronously to simulate massive enterprise data collection pipelines.

The finalized microservices stack is fully active, tested, and ready for review on the public AWS EC2 instance.

Table 1: Consolidated Microservices Architecture and Cloud Specifications

Service Name	Docker Suffix / Container Name	Internal Port	Public Port (EC2)
Postgres DB	postgres_294	5432	<i>Shielded</i>
RabbitMQ Broker	rabbitmq_294	5672 / 15672	15672 (Admin Dashboard)
Auth Microservice	auth_service_294	5001	5001 (Register/Login)
Event Microservice	event_service_294	5002	5002 (Secure Events)
Notification Daemon	notification_service_294	5004	<i>Background Consumer</i>